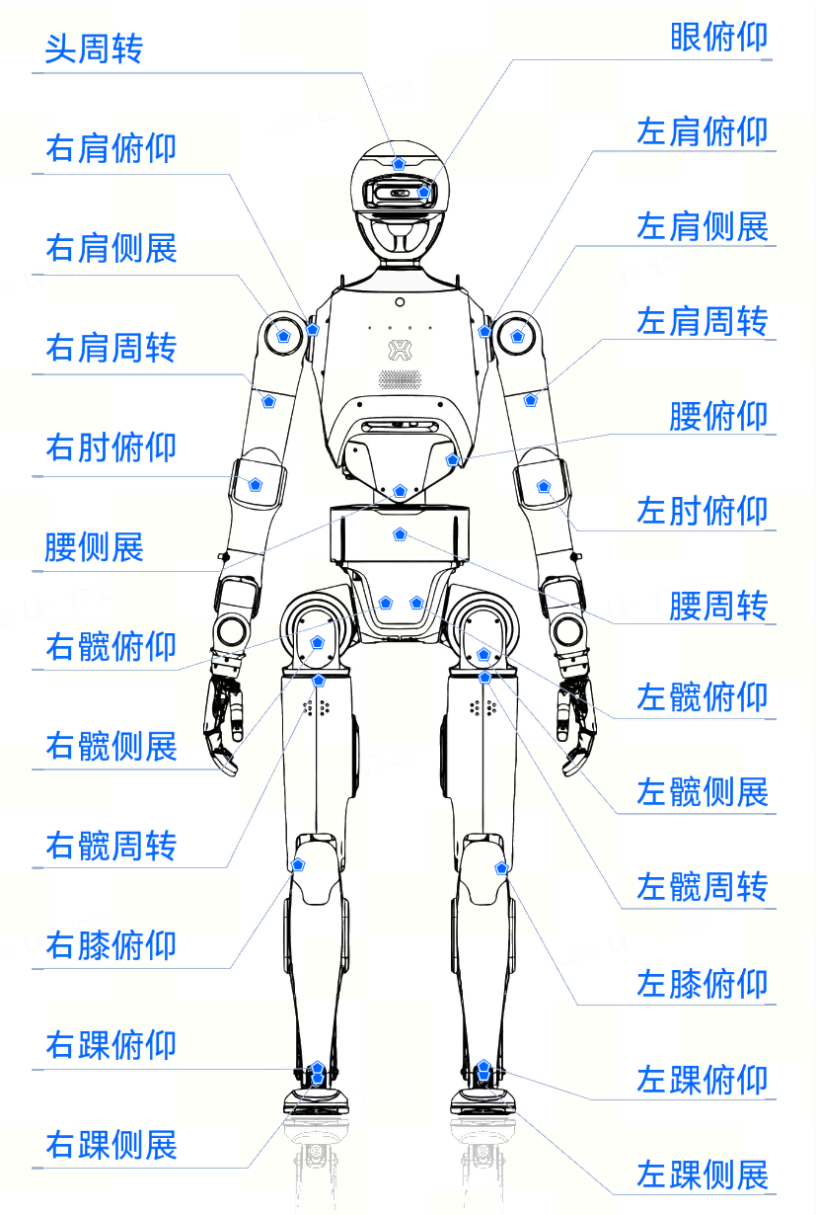
天工行者DEX SDK 文档（260814）

1. 关于天工行者DEX

1.1 全身关节参数



注：具体关节id参照 - 4.1.1全身关节id定义

所属部位	名称	运动范围（°）	扭矩系数（Nm/A）	最大转速（rpm）	KP/KD范围	额定力矩（Nm）	峰值力矩（Nm）	关节名称

头部	头关节偏航 Head Yaw	-60°~+60°	2.16	72	Kp：0~2000 kd：0~300	1.6	4.6	1
	眼关节俯仰 Head Pitch	-15°~+58°	2.16	72	Kp：0~2000 kd：0~300	1.6	4.6	2
腰部	腰关节俯仰 Waist Pitch	-30°~+50°	2.44	132	Kp：0~2000 kd：0~300	35	150	31
	腰关节翻滚 Waist Roll	-25°~+25°	2.44	132	Kp：0~2000 kd：0~300	35	150	32
	腰关节偏航 Waist Yaw	-150°~+185°	3.2	88	Kp：0~2000 kd：0~300	34	91	33
左臂	左肩关节俯仰 Left Shoulder Pitch	-163°~+163°	4.6	69	Kp：0~2000 kd：0~300	30	90	11
	左肩关节翻滚 Left Shoulder Roll	-11°~+191°	4.6	69	Kp：0~2000 kd：0~300	30	90	12
	左肩关节偏航 Left Shoulder Yaw	-163°~+163°	2.7	109	Kp：0~2000 kd：0~300	20	50	13
	左肘关节俯仰 Left Elbow Pitch	-146°~+11°	2.7	109	Kp：0~2000 kd：0~300	20	50	14
	左腕关节偏航 Left Wrist Yaw	-163°~+163°	1.7	117	Kp：0~2000 kd：0~300	7	25	15
	左腕关节俯仰 Left Wrist Pitch	-76°~+76°	1.7	117	Kp：0~2000 kd：0~300	7	25	16

	左腕关节翻滚 Left Wrist Roll	-76°~+76°	1.7	117	Kp：0~2000 kd：0~300	7	25	17
右臂	右肩关节俯仰 Right Shoulder Pitch	-163°~+163°	4.6	69	Kp：0~2000 kd：0~300	30	90	21
	右肩关节翻滚 Right Shoulder Roll	-191°~+11°	4.6	69	Kp：0~2000 kd：0~300	30	90	22
	右肩关节偏航 Right Shoulder Yaw	-163°~+163°	2.7	109	Kp：0~2000 kd：0~300	20	50	23
	右肘关节俯仰 Right Elbow Pitch	-146°~+11°	2.7	109	Kp：0~2000 kd：0~300	20	50	24
	右腕关节偏航 Right Wrist Yaw	-163°~+163°	1.7	117	Kp：0~2000 kd：0~300	7	25	25
	右腕关节俯仰 Right Wrist Pitch	-76°~+76°	1.7	117	Kp：0~2000 kd：0~300	7	25	26
	右腕关节翻滚 Right Wrist Roll	-76°~+76°	1.7	117	Kp：0~2000 kd：0~300	7	25	27
左腿部	髋关节俯仰 Hip Pitch	-175~+160°	2.1	160	Kp：0~2000 kd：0~300	60	235	51
	髋关节翻滚 Hip Roll	-24°~+150°	2.1	160	Kp：0~2000 kd：0~300	60	235	52
	髋关节偏航 Hip Yaw	-80°~+255°	2.44	132	Kp：0~2000 kd：0~300	35	150	53

	膝关节俯仰 Knee Pitch	-5°~+145°	3.26	106	Kp： 0~2000 kd：0~300	86	400	54
	踝关节俯仰 Ankle Pitch	-68°~+30°	2.6	135	Kp： 0~2000 kd：0~300	17.5	55	55
	踝关节翻滚 Ankle Roll	-30°~+30°	2.6	135	Kp： 0~2000 kd：0~300	17.5	55	56
右腿部	髋关节俯仰 Hip Pitch	-175~+160 °	2.1	160	Kp： 0~2000 kd：0~300	60	235	61
	髋关节翻滚 Hip Roll	-24°~+150°	2.1	160	Kp： 0~2000 kd：0~300	60	235	62
	髋关节偏航 Hip Yaw	-80°~+255°	2.44	132	Kp： 0~2000 kd：0~300	35	150	63
	膝关节俯仰 Knee Pitch	-5°~+145°	3.26	106	Kp： 0~2000 kd：0~300	86	400	64
	踝关节俯仰 Ankle Pitch	-68°~+30°	2.6	135	Kp： 0~2000 kd：0~300	17.5	55	65
	踝关节翻滚 Ankle Roll	-30°~+30°	2.6	135	Kp： 0~2000 kd：0~300	17.5	55	66

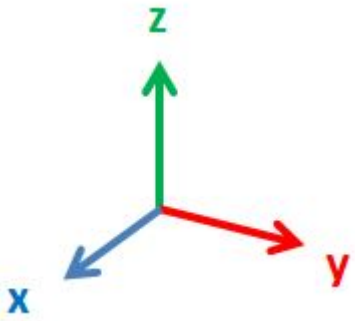
1.2 整机urdf，坐标系，关节旋转轴与关节零点

1.2.1 坐标系

a. 整机坐标系-X轴（Roll） 以机器人前方为正方向；

b. 整机坐标系-Y轴（Pitch） 以机器人左方为正方向；

c. 整机坐标系-Z轴（Yaw） 以机器人上方为正方向；



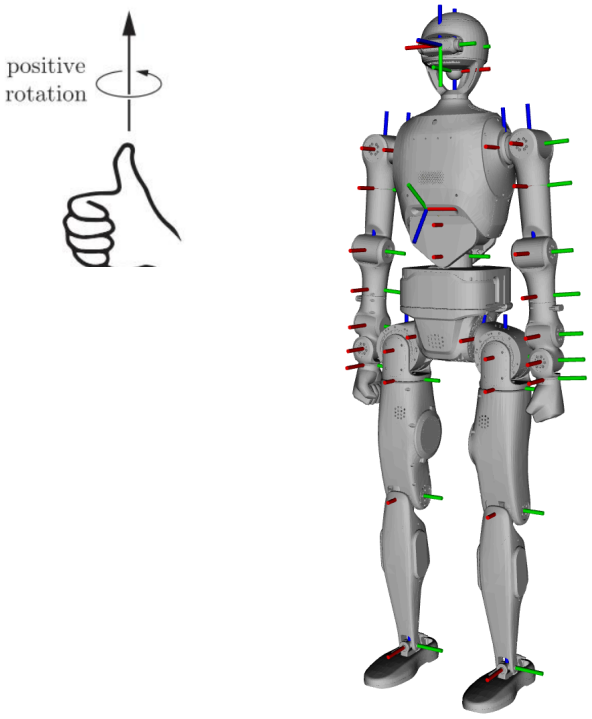
1.2.2 旋转轴 & 正方向

a. 机器人每个旋转轴的轴线，按照下图的状态为基准，和机器人整机坐标系的正方向对应去寻找旋转轴的正方向；

b. 举例：

i. 比如腰部关节的轴线是竖直的，和整机Z轴（Yaw）靠近，所以腰部关节的正方向是向上，同构右手定则能够确认关节的选装正方向；

ii. 比如腿部髋关节Pitch的轴线，和整机Y轴（Pitch）靠近，所以髋关节pitch的正方向是向左，同构右手定则能够确认关节的选装正方向。



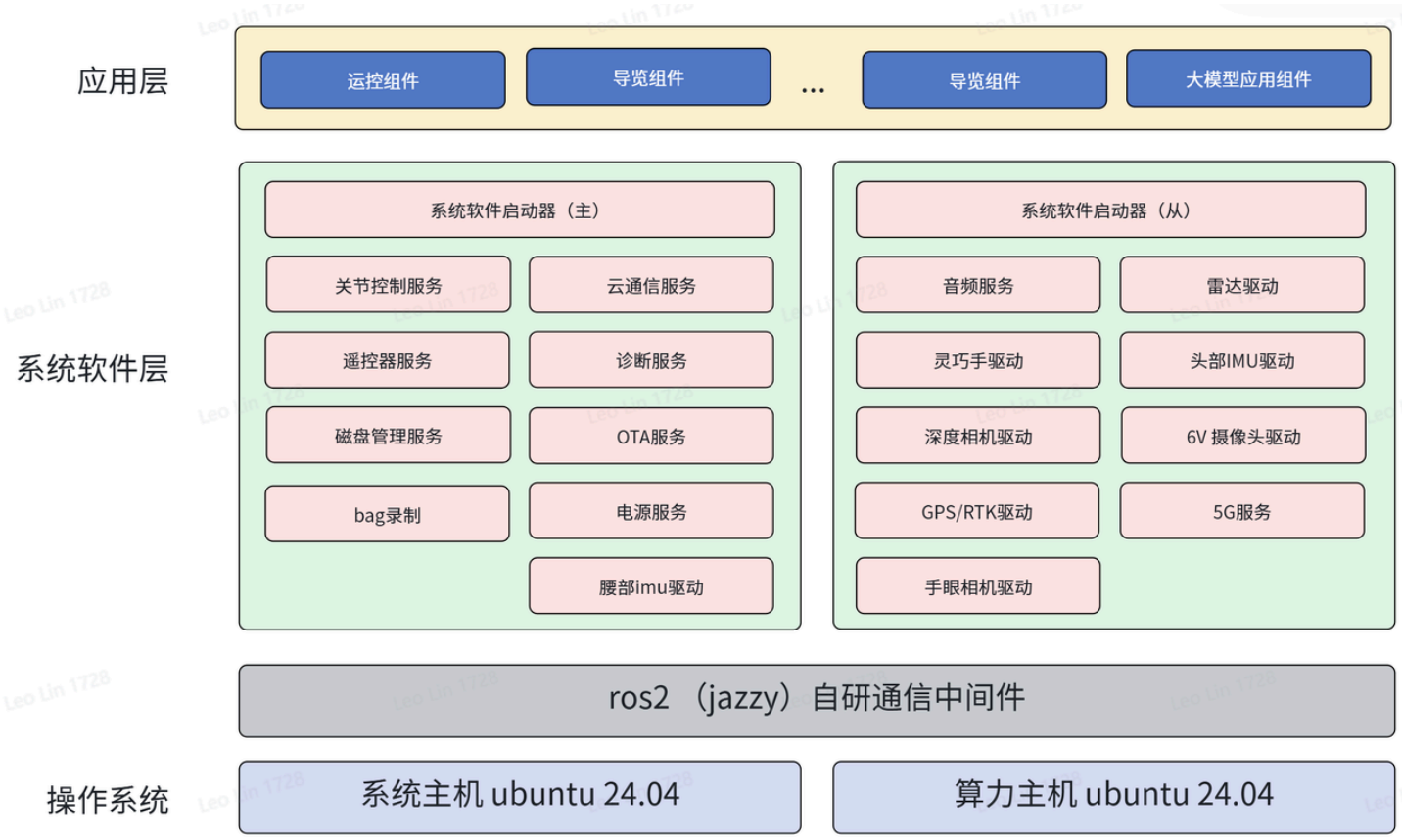
2. SDK 概述

天工行者DEX SDK由深圳市优必选科技股份有限公司开发，提供了丰富的接口，涵盖各部位电机控制、IMU（惯性测量单元）、语音、电池、遥控器等。可用于编写和部署机器人应用程序，旨在帮助开发人员快速灵活地构建自己的应用程序来精确控制和使用机器人，以满足在不同应用场景下的需求。

示例代码库

https://github.com/Open-X-Humanoid/xhumanoid_sdk

2.1 基本框架 - ROS2



2.2 系统环境依赖

为了确保最佳的开发体验和兼容性，建议在Ubuntu 24.04系统下进行开发。当前暂不支持在Mac和Windows系统下开发。

2.3 运行环境说明

2.3.1 系统软件常用路径

代码块

```
#arm本体安装路径
/home/nvidia/xos

#arm全局配置参数路径
/home/nvidia/data/param

#arm日志目录
/home/nvidia/logs
```

2.3.2 注意事项

- 不建议在机器人系统主机和算力主机运行 apt upgrade 命令，该命令有可能会更新系统软件的依赖库导致机器人运行异常
- 机器人系统算力主机需要使用user用户（nvidia）启动ros2节点，否则ros2消息会无法通信

2.4 开发环境说明

2.4.1 网络配置

- 机器人后背颈部的网口，连接到机器人内部的交换机上，外部连接的设备需设置ip为192.168.41.(3-255)，mtu为9000进行连接。
- 机器人后背腰部的网口，连接到机器人的算力主机上，需设置ip为192.168.10.(2-255)，mtu为9000进行连接。

3. 系统软件工具

3.1 proc_manager

proc_manager 是所有本体软件启动管理器，可通过systemd的方式完成系统相关组件的开关。

3.1.1 开关方式

自启动功能默认被禁用，使用以下方法开启、关闭:

代码块

```
#开启系统软件自启动
sudo systemctl enable proc_manager.service
#关闭系统软件自启动
sudo systemctl disable proc_manager.service
#开启系统软件
sudo systemctl start proc_manager.service
#关闭系统软件
sudo systemctl stop proc_manager.service
#查看系统组件状态
sudo systemctl status proc_manager.service
```

3.1.2 配置文件

3.1.2.1 配置文件路径：

`/home/nvidia/xos/proc_manager/share/proc_manager/param/proc_manager.json`

3.1.2.2 配置项说明：

代码块

```
{
  ...
  "proc": [
    {
      "name": "power_board",                # 服务名称
      "enable": true,                       # true：生效，false：忽略
      "shell": "ros2 launch power_board power_board.py", # shell：要执行的shell命令
      "boot_start": true,                   # 开机是否自动启动，true：是，false：否
      "ready_topic": "power_board_state"    # 服务状态ready的topic，等待该topic信号标识服务已启动完成
    },{
      "name": "usb_sbus",
      "enable": true,
      "shell": "ros2 launch usb_sbus usb_sbus.launch.py",
      "boot_start": true
    },{
      "name": "body_control",
      "enable": true,
      "shell": "ros2 launch body_control body_no_sbus.launch.py",
      "boot_start": false,
      "ready_topic": "bodycontrol_state",
      "ready_timeout": 20
    },{
      "name": "motion",
      "enable": true,
      "shell": "ros2 launch motion_control motion.py"
    }
  ],
  ...
}
```

3.2 序列号工具 xsys

3.2.1 ros2接口

3.2.1.1 获取机器人序列号：

功能说明	获取机器人序列号。
控制方式	service
消息名称	`/xsys/get_serial_number`
数据类型	`xsys_msgs/srv/GetSerialNumber.srv`
数据格式	代码块 1 # xsys_msgs/srv/GetSerialNumber.srv 2 3 string SERVICE_NAME = /xsys/get_serial_number 4 5 --- 6 string serial_number
命令行示例	ros2 service call /xsys/get_serial_number

3.3 磁盘管理

3.3.1 disk_manage

磁盘空间管理程序，作为一个systemd系统服务运行，可以按照配置进行磁盘空间的管理，防止日志文件长期堆积浪费磁盘空间，甚至导致磁盘被占满后引发系统异常；

配置文件格式与支持的配置项，详见3.3.3章节

程序职责：

- 程序运行日志管理
- rosbag文件管理
- 磁盘空间清理
  - 清理指定目录内的文件（配置文件中list指定的目录列表）
  - 循环删除 最旧的程序运行日志和最旧的rosbag 直到磁盘空间占用 降至 所设阈值以下

3.3.2 xlog

日志库，提供如下特性：

- 提供异步日志写入服务
- 提供日志轮转功能
- 与disk_manage的无缝协作

使用方法详见 example/xlog_demo

3.3.3 配置文件说明

代码块

```
{
    //磁盘空间占用超限后的目录清理列表
    "list": [
        {
            "path_name": "/mnt/code/file_manage/tmp1",
            // path_name下，受保护的子项，这些子项不会被删除
            "sub_path_name": [
                "tmp2_1",
                "tmp2_3",
                "log"
            ]
        }
    ],
    //日志文件和rosbag文件管理
    "log_file": [
        {
            // 程序日志 默认存储目录
            "path_name": "/home/ubuntu/logs",
            // 支持 time size mixed三种模式
            // time: 精度天，超过时间阈值的日志文件将被删除
            // size: 按照磁盘占用量管理，目录大小超过限制后，删除最旧文件
            // mixed: 采用 time 和 size 的混合策略
            "mode": "time",
            "save_day": 3,
        },
        {
            // rosbag 默认存储目录
            "path_name": "/home/ubuntu/bags",
            // 对于robbag文件采用混合策略
            "mode": "mixed",
            "save_day": 3,
            // 单位:  MB
            "storage_size": 65536
        }
    ],
    "FlashPath": "/",
    //磁盘占用阈值，超过此阈值后，将会删除
    "FlashUsedMax": 90,
    //xlog库日志回滚配置，对使用xlog的程序有效；支持按照logger_id独立配置
    "log_rotation": {
        // 默认的回滚配置，按照大小进行管理
```

```
        "default": {
          // 支持的模式:
          //    size: 按照大小进行管理，可以配置单个文件的最大大小和总大小
          //    time:  按照时长进程管理
          "mode" : "size",
          // 单个文件最大大小，超过限制后生成一个新的日志文件,对size和time模式均有效
          "max_file_size": 10485760,
          // 大小管理策略，可以配置总的大小限制，默认100MB
          "size_policy": {
            "total_limit": 104857600
          },
          // 时长管理策略，可以配置日志默认的保存天数
          "time_policy": {
            "duration": "3d"
          }
        },
        // 针对具体logger_id的配置策略，若不存在，使用上面的默认配置
        "xlog_test": {
          "max_file_size": 60000,
          "size_policy": {
            "total_limit": 300000
          }
        }
    }
}

```

3.4 遥控器

3.4.1 功能说明

3.4.2 ros2接口

功能说明	标准 ROS2 Joy 消息，包含 12 个轴数据
控制方式	ros2 topic
消息名称	/sbus_data
数据类型	sensor_msgs/msg/Joy
数据格式	代码块 1 # Reports the state of a joysticks axes and buttons.

```
std_msgs/Header header
float32[] axes   # 12个浮点值，范围 [-1.0, 1.0]
int32[] buttons  # 空（不使用）
```

命令行示例	ros2 topic echo /sbus_data

功能说明	自定义 SBUS 数据消息，包含按键事件和状态
控制方式	ros2 topic
消息名称	/sbus_data/event
数据类型	bodyctrl_msgs/msg/SbusData
数据格式	代码块

```
# 按键事件常量定义
int32 KEY_NONE=0
int32 KEY_A_UP=1                # A按键松开
int32 KEY_A_DOWN=2              # A按键按下
int32 KEY_B_UP=3                # B按键松开
int32 KEY_B_DOWN=4              # B按键按下
int32 KEY_C_UP=5                # C按键松开
int32 KEY_C_DOWN=6              # C按键按下
int32 KEY_D_UP=7                # D按键松开
int32 KEY_D_DOWN=8              # D按键按下
int32 KEY_E_UP=9                # E上拨
int32 KEY_E_MID=10              # E中间位置
int32 KEY_E_DOWN=11             # E下拨
int32 KEY_F_UP=12               # F上拨
int32 KEY_F_MID=13              # F中间位置
int32 KEY_F_DOWN=14             # F下拨
int32 KEY_G_LEFT=15             # G左拨
int32 KEY_G_MID=16              # G中间位置
int32 KEY_G_RIGHT=17            # G右拨
int32 KEY_H_LEFT=18             # H左拨
int32 KEY_H_MID=19              # H中间位置
int32 KEY_H_RIGHT=20            # H右拨

std_msgs/Header header          # ROS2消息头

# 按键事件：值对应上述KEY_*常量
int32 key_event_new             # 当前按键事件（变化后的状态）
int32 key_event_old             # 之前的按键事件（变化前的状态）

```

```
# 按键状态：-1=松开/复位，0=中间位置，1=按下/拨动到一端，2=拨动到另一端
int8 button_a                    # A按键状态
int8 button_b                    # B按键状态
int8 button_c                    # C按键状态
int8 button_d                    # D按键状态
int8 button_e                    # E三档开关状态
int8 button_f                    # F三档开关状态
int8 button_g                    # G左右拨动开关状态
int8 button_h                    # H左右拨动开关状态

# 摇杆数据：范围[-1.0, 1.0]，直接透传不滤波
float32 x1                       # 左摇杆X轴：左(-1) ↔ 右(+1)
float32 y1                       # 左摇杆Y轴：下(-1) ↔ 上(+1)
float32 x2                       # 右摇杆X轴：左(-1) ↔ 右(+1)
float32 y2                       # 右摇杆Y轴：下(-1) ↔ 上(+1)
```

命令行示例	ros2 topic echo /sbus_data/event

3.5 电源控制

本模块包含电源板上位机，灯带上位机，实现功能包括监测主电池、副电池状态，监测全身供电状态和控制电源板的命令，急停按键状态、控制灯带灯功能。其接口也包括三类（电源板，电池，灯带），五个接口，即：

i. 电源板状态接口

ii. 电源板控制接口

iii. 电源板急停状态接口

iv. 电池状态接口

v. 灯带控制接口

3.5.1 电源板状态ros2接口

代码块

```
#前置命令：
source ~/xos/setup.bash
```

功能说明	自定义电源板 数据消息，包含四肢电压电流等模拟量和版本信息等
控制方式	ros2 topic

消息名称	/power/board/status
数据类型	bodyctrl_msgs/msg/PowerStatus.msg
数据格式	代码块

```
# @brief 电源板电压、电流、电量、温度等信息

std_msgs/Header header

# temperature information
float32 waist_temp # 腰部MOS温度
float32 arm_a_temp # 臂A MOS温度
float32 arm_b_temp # 臂B MOS温度
float32 leg_a_temp # 腿A MOS温度
float32 leg_b_temp # 腿B MOS温度

float32 waist_temp_max # 腰部MOS温度最大值
float32 arm_a_temp_max # 臂A MOS温度最大值
float32 arm_b_temp_max # 臂B MOS温度最大值
float32 leg_a_temp_max # 腿A MOS温度最大值
float32 leg_b_temp_max # 腿B MOS温度最大值

float32 waist_temp_min # 腰部MOS温度最小值
float32 arm_a_temp_min # 臂A MOS温度最小值
float32 arm_b_temp_min # 臂B MOS温度最小值
float32 leg_a_temp_min # 腿A MOS温度最小值
float32 leg_b_temp_min # 腿B MOS温度最小值

# current information
float32 arm_a_curr # 臂A电流
float32 arm_b_curr # 臂B电流
float32 leg_a_curr # 腿A电流
float32 leg_b_curr # 腿B电流
float32 waist_curr # 腰部电流
float32 head_curr  # 头部电流

float32 arm_a_curr_max # 臂A电流最大值
float32 arm_b_curr_max # 臂B电流最大值
float32 leg_a_curr_max # 腿A电流最大值
float32 leg_b_curr_max # 腿B电流最大值
float32 waist_curr_max # 腰部电流最大值
float32 head_curr_max  # 头部电流最大值

float32 arm_a_curr_min # 臂A电流最小值
float32 arm_b_curr_min # 臂B电流最小值
float32 leg_a_curr_min # 腿A电流最小值
float32 leg_b_curr_min # 腿B电流最小值
```

```
float32 waist_curr_min # 腰部电流最小值
float32 head_curr_min  # 头部电流最小值

float32 arm_a_volt #臂A电压
float32 arm_b_volt #臂B电压
float32 leg_a_volt #腿A电压
float32 leg_b_volt #腿B电压
float32 waist_volt #腰部电压
float32 bus_volt   #母线电压

float32 arm_a_volt_max #臂A电压最大值
float32 arm_b_volt_max #臂B电压最大值
float32 leg_a_volt_max #腿A电压最大值
float32 leg_b_volt_max #腿B电压最大值
float32 waist_volt_max #腰部电压最大值
float32 bus_volt_max #母线电压最大值

float32 arm_a_volt_min #臂A电压最小值
float32 arm_b_volt_min #臂B电压最小值
float32 leg_a_volt_min #腿A电压最小值
float32 leg_b_volt_min #腿B电压最小值
float32 waist_volt_min #腰部电压最小值
float32 bus_volt_min #母线电压最小值

# version information
string software_version # 软件版本
string hardware_version # 硬件版本

```

命令行示例	ros2 topic echo /power/board/status

3.5.2 电源板控制ros2接口

代码块

```
#前置命令：
source ~/xos/setup.bash
```

功能说明	自定义电源板 控制命令
控制方式	ros2 topic
消息名称	/power/board/ctrl

数据类型	bodyctrl_msgs/msg/PowerBoardCtrl.msg
数据格式	代码块 1 std_msgs/Header header 2 int32 cmd
命令行示例	ros2 topic echo /power/board/ctrl

3.5.3 电源板急停状态ros2接口

代码块

```
#前置命令：
source ~/xos/setup.bash
```

功能说明	自定义电源板 关键状态信息，包含电源急停按键状态，关机命令触发状态等
控制方式	ros2 topic
消息名称	/power/board/key_status
数据类型	bodyctrl_msgs/msg/PowerBoardKeyStatus.msg
数据格式	代码块 1 # @brief 电源板按键状态 2 3 std_msgs/Header header 4 5 # power board status 6 uint32 work_time 7 std_msgs/Bool is_estop #机器人背部急停信号 急停按下，该值为真 8 std_msgs/Bool is_remote_estop #遥控器急停信号 遥控急停按下，该值为真 9 std_msgs/Bool is_power_on #电源48V供电信号 48V供电打开，该值为真 10 std_msgs/Bool is_shutting_down #即将关机的状态 接收到关机信号，该值为真 11 std_msgs/Bool is_battery_unlock #电池舵机解锁专题 非换电状态且舵机解锁，该值为真

命令行示例	ros2 topic echo /power/board/key_status

3.5.4 电池状态ros2接口

代码块

```
#前置命令：
source ~/xos/setup.bash
```

功能说明	自定义电池信息，包含主电池和副电池的供电情况，健康状态等信息
控制方式	ros2 topic
消息名称	/power/battery/status
数据类型	bodyctrl_msgs/msg/PowerBatteryStatus.msg

数据格式	代码块
	`# @brief 电源板电压、电流、电量、温度等信息`
	
	`std_msgs/Header header`
	
	`# battery status information`
	`int32 battery_installed  # 0x00都不存在，0x01小电池存在，0x02大电池存在，0x03大小电池都存在`
	`int32 battery_working    # 0x01大电池，0x10小电池`
	
	`float32 master_battery_voltage # 电池电压信息`
	`float32 master_battery_current # 电池电流信息`
	`float32 master_battery_power   # 电池电量`
	`float32 master_battery_pcbtemp # 电池pcb板温度`
	
	`float32 little_battery_voltage # 小电池电压信息`
	`float32 little_battery_current # 小电池电流信息`
	`float32 little_battery_power   # 小电池电量`
	`float32 little_battery_pcbtemp # 小电池pcb板温度`
	
	`int8 pg12a # 00低，01高—预留，未赋值`
	`int8 pg12b # 00低，01高—预留，未赋值`
	`int8 pg12c # 00低，01高—预留，未赋值`
	`int8 pg12d # 00低，01高—预留，未赋值`
	`int8 pg5cd # 00低，01高—预留，未赋值`

```
int8 pg5ab # 00低，01高——预留，未赋值

int8 pgrdc2    # 00低，01高 ——预留，未赋值
int8 pgrdc1    # 00低，01高 ——预留，未赋值
int8 pgheader   # 00低，01高 ——预留，未赋值
int8 pgbutton2 # 00低，01高 ——预留，未赋值
```

命令行示例	ros2 topic echo /power/battery/status

3.5.5 灯带控制命令ros2接口

功能说明	自定义灯带控制命令
控制方式	ros2 topic
消息名称	/xsys/light/ctrl
数据类型	bodyctrl_msgs/msg/LightCtrl.msg
数据格式	代码块

```
string TOPIC_NAME = /xsys/light/ctrl

# 系统
int32 CMD_SYSTEM_SHUTTING = 0    # 关机中
int32 CMD_SYSTEM_ERROR_OCCUR = 10  # 故障
int32 CMD_SYSTEM_ERROR_CLEAR = 11  # 故障消除
int32 CMD_SYSTEM_WARN_OCCUR = 12   # 告警
int32 CMD_SYSTEM_WARN_CLEAR = 13   # 告警消除
int32 CMD_SYSTEM_SERVICE_WAIT = 20   # 服务等待中
int32 CMD_SYSTEM_SERVICE_START = 21  # 服务启动中
int32 CMD_SYSTEM_SERVICE_READY = 22     # 服务就绪
int32 CMD_SYSTEM_SERVICE_FAILED = 23    # 服务启动失败
int32 CMD_SYSTEM_STANDBY = 99   # 系统待机

# 基础组件
int32 CMD_OTA_QUIT = 100   # 升级退出
int32 CMD_OTA_START = 101  # 升级开始

# 电源
int32 CMD_POWER_QUIT = 200                    # 电量状态退出
int32 CMD_POWER_BATTERY_NORMAL = 201          # 放电中，电量充足
```

```
int32 CMD_POWER_BATTERY_LOW = 202           # 放电中，低电量
int32 CMD_POWER_BATTERY_CRITICAL = 203      # 放电中，电量危急
int32 CMD_POWER_CHARGING = 210              # 充电中
int32 CMD_POWER_CHARGING_FULL = 211         # 充电中，已充满
int32 CMD_POWER_BACKUP_NORMAL = 220         # 备份电池放电中

# 交互
int32 CMD_CHAT_QUIT = 300                   # 对话结束，已休眠
int32 CMD_CHAT_WAKEUP = 301                 # 对话开始，已唤醒
int32 CMD_CHAT_ASR = 310                    # 拾音中
int32 CMD_CHAT_LLM = 311                    # 推理中
int32 CMD_CHAT_TTS = 312                    # 合成中
int32 CMD_CHAT_PLAY = 313                   # 应答播报中
int32 CMD_CHAT_NET_OFFLINE = 320            # 断网
int32 CMD_CHAT_NET_ONLINE = 321             # 网络恢复

# 运动
int32 CMD_MOTION_QUIT = 400                 # 运动退出
int32 CMD_MOTION_RUNNING = 401              # 奔跑中

std_msgs/Header header
int32  cmd              # 灯效定义
int8[] data             # 可选的载荷数据
string caller_id        # 调用方身份
string caller_msg       # 调用方提供的描述信息 —— 基于什么原因更新了灯效，用作日志审计目的
```

命令行示例	ros2 topic echo /xsys/light/ctrl

4. 身体控制

4.1 关节

4.1.1 全身关节id定义

- 说明：头部、臂部、腰部和腿部的电机的命名ID定义如下。

```
代码块 head 00 + id
  uint16 MOTOR_HEAD_1 = 1
  uint16 MOTOR_HEAD_2 = 2

  # arm-left 10 + id
  uint16 MOTOR_ARM_LEFT_1 = 11
  uint16 MOTOR_ARM_LEFT_2 = 12
  uint16 MOTOR_ARM_LEFT_3 = 13
  uint16 MOTOR_ARM_LEFT_4 = 14
  uint16 MOTOR_ARM_LEFT_5 = 15
  uint16 MOTOR_ARM_LEFT_6 = 16
  uint16 MOTOR_ARM_LEFT_7 = 17

  # arm-right 20 + id
  uint16 MOTOR_ARM_RIGHT_1 = 21
  uint16 MOTOR_ARM_RIGHT_2 = 22
  uint16 MOTOR_ARM_RIGHT_3 = 23
  uint16 MOTOR_ARM_RIGHT_4 = 24
  uint16 MOTOR_ARM_RIGHT_5 = 25
  uint16 MOTOR_ARM_RIGHT_6 = 26
  uint16 MOTOR_ARM_RIGHT_7 = 27

  # body (including waist)
  uint16 MOTOR_WAIST_1 = 31
  uint16 MOTOR_WAIST_2 = 32
  uint16 MOTOR_WAIST_3 = 33

  # 40 rfu

  # leg-left 50 + id
  uint16 MOTOR_LEG_LEFT_1 = 51
  uint16 MOTOR_LEG_LEFT_2 = 52
  uint16 MOTOR_LEG_LEFT_3 = 53
  uint16 MOTOR_LEG_LEFT_4 = 54
  uint16 MOTOR_LEG_LEFT_5 = 55
  uint16 MOTOR_LEG_LEFT_6 = 56

  # leg-right 60 + id
  uint16 MOTOR_LEG_RIGHT_1 = 61
  uint16 MOTOR_LEG_RIGHT_2 = 62
  uint16 MOTOR_LEG_RIGHT_3 = 63
  uint16 MOTOR_LEG_RIGHT_4 = 64
  uint16 MOTOR_LEG_RIGHT_5 = 65
  uint16 MOTOR_LEG_RIGHT_6 = 66
```

4.1.2 获取状态信息

说明	获取机器人所有的状态信息。
接口类型	topic
消息名称	/robot_state
数据类型	ros2_bridge_msgs/msg/RobotState
数据格式	代码块 1 std_msgs/Header header 2 3 ros2_bridge_msgs/HeadStatus head 4 ros2_bridge_msgs/WaistStatus waist 5 ros2_bridge_msgs/LegStatus leg 6 ros2_bridge_msgs/ArmStatus arm 7 ros2_bridge_msgs/ImuStatus imu
头部关节状态	代码块 1 # HeadStatus.msg 2 # 头部关节状态消息 3 4 std_msgs/Header header 5 uint8 label # 调用标签 6 uint8 reserved # 保留字段，暂时未用 7 ros2_bridge_msgs/MotorStatus[] status
腰部关节状态	代码块 1 # WaistStatus.msg 2 # 腰部状态消息 3 4 std_msgs/Header header 5 uint8 label # 调用标签 6 uint8 reserved # 保留字段，暂时未用 7 ros2_bridge_msgs/MotorStatus[] status
手臂关节状态	代码块 1 # ArmStatus.msg 2 # 手臂状态消息 3 4 std_msgs/Header header 5 uint8 label # 调用标签

```
uint8 reserved # 保留字段，暂时未用
ros2_bridge_msgs/MotorStatus[] status
```

腿部关节状态

代码块

```
# LegStatus.msg
# 腿部状态消息

std_msgs/Header header
uint8 label # 调用标签
uint8 reserved # 保留字段，暂时未用
ros2_bridge_msgs/MotorStatus[] status
```

MotorStatus

代码块

```
# MotorStatus.msg
# 电机状态消息

uint16 name
float64 pos
float64 speed
float64 current
float64 temperature
float64 mos_temperature
uint32 error
```

IMU状态

代码块

```
# ImuStatus.msg
# IMU 状态消息

std_msgs/Header header
uint8 label # 调用标签
uint8 reserved # 保留字段，暂时未用

# 四元数
float64 qx
float64 qy
float64 qz
float64 qw

# 欧拉角（度）
float64 roll
float64 pitch
float64 yaw
```

	代码块 ``` # 角速度 (rad/s) float64 wx float64 wy float64 wz # 线性加速度 (m/s^2) float64 ax float64 ay float64 az # 协方差矩阵（按行展开） float64[9] orientation_covariance float64[9] angular_velocity_covariance float64[9] linear_acceleration_covariance # 错误码 uint32 error ```
命令行示例	代码块 ``` ros2 topic echo /robot_state ```

4.1.3 控制话题

4.1.3.1 双臂关节

接口类型	topic
消息名称	/arm/cmd
数据类型	ros2_bridge_msgs/msg/ArmCtrl
数据格式	代码块 ``` # ArmCtrl.msg # 手臂控制消息 std_msgs/Header header uint8 mode # 控制模式: 0=位置模式, 1=力位混合模式, 2=速度模式, 3=标零模式 4=距离模式 5=电流模式 uint8 label # 调用标签（选填，或者填写101-255之间任意值） uint8 reserved # 保留字段，暂时未用 ```

```
ros2_bridge_msgs/MotorCtrl[] ctrl
```

MotorCtrl

代码块

```
# MotorCtrl.msg
# 电机控制消息

uint16 name
float64 kp      # 比例增益（力位混合模式使用）
float64 kd      # 微分增益（力位混合模式使用）
float64 pos     # 期望位置（rad）（位置模式和力位混合模式使用）
float64 spd     # 期望速度（rad/s）（位置模式、速度模式和力位混合模式使用）
float64 tor     # 前馈力矩（力位混合模式使用）
float64 cur     # 最大电流（A）（位置模式和速度模式使用）
string joint_ids  # 关节ID字符串，逗号分隔，如 "1,2,3"（标零模式使用）
```

命令行示例

代码块

```
# 手臂关节11使用力位混合控制
ros2 topic pub --once /arm/cmd ros2_bridge_msgs/msg/ArmCtrl "{
  header: {stamp: {sec: 0, nanosec: 0}, frame_id: 'arm'},
  mode: 1,
  label: 101,
  ctrl: [
    {name: 11, kp: 50.0, kd: 2.0, pos: 0.5, spd: 0, tor: 0}
  ]
}"

# 手臂关节11使用位置模式控制
ros2 topic pub --once /arm/cmd ros2_bridge_msgs/msg/ArmCtrl "{
  header: {stamp: {sec: 0, nanosec: 0}, frame_id: 'arm'},
  mode: 0,
  label: 101,
  ctrl: [
    {name: 11, pos: 0.5, spd: 0.2, cur: 10}
  ]
}"
```

4.1.3.2 腿部关节

接口类型	topic
消息名称	/leg/cmd
数据类型	ros2_bridge_msgs/msg/LegCtrl

数据格式	代码块
	`# LegCtrl.msg`
	`# 腿部控制消息`
	
	`std_msgs/Header header`
	`uint8 mode  # 控制模式: 0=位置模式, 1=力位混合模式, 2=速度模式, 3=标零模式  4=距离模式`
	`uint8 label # 调用标签（选填，或者填写101-255之间任意值）`
	`uint8 reserved # 保留字段，暂时未用`
	`ros2_bridge_msgs/MotorCtrl[] ctrl`

MotorCtrl	代码块
	`# MotorCtrl.msg`
	`# 电机控制消息`
	
	`uint16 name`
	`float64 kp  # 比例增益（力位混合模式使用）`
	`float64 kd  # 微分增益（力位混合模式使用）`
	`float64 pos  # 期望位置（rad）（位置模式和力位混合模式使用）`
	`float64 spd  # 期望速度（rad/s）（位置模式、速度模式和力位混合模式使用）`
	`float64 tor  # 前馈力矩（力位混合模式使用）`
	`float64 cur  # 最大电流（A）（位置模式和速度模式使用）`
	`string joint_ids  # 关节ID字符串，逗号分隔，如 "1,2,3"（标零模式使用）`

命令行示例	代码块
	`# 腿部关节51使用力位混合控制`
	`ros2 topic pub --once /leg/cmd ros2_bridge_msgs/msg/LegCtrl "{`
	`  header: {stamp: {sec: 0, nanosec: 0}, frame_id: 'leg'},`

```
    mode: 1,
    label: 101,
    ctrl: [
      {name: 51, kp: 50.0, kd: 2.0, pos: 0.5, spd: 0, tor:
0}
    ]
}"


# 腿部关节51使用位置模式控制
ros2 topic pub --once /leg/cmd
ros2_bridge_msgs/msg/LegCtrl "{
    header: {stamp: {sec: 0, nanosec: 0}, frame_id: 'leg'},
    mode: 0,
    label: 101,
    ctrl: [
      {name: 51, pos: 0.5, spd: 0.2, cur: 10}
    ]
}"
```

4.1.3.3 腰部关节

接口类型	topic
消息名称	/waist/cmd
数据类型	ros2_bridge_msgs/msg/WaistCtrl

数据格式	代码块
	`# WaistCtrl.msg`
	`# 腰部控制消息`
	
	`std_msgs/Header header`
	`uint8 mode  # 控制模式：0=位置模式，1=力位混合模式，2=速度模式，3=标零模式  4=距离模式`
	`uint8 label # 调用标签（选填，或者填写101-255之间任意值）`
	`uint8 reserved # 保留字段，暂时未用`
	`ros2_bridge_msgs/MotorCtrl[] ctrl`

MotorCtrl	代码块

```
# MotorCtrl.msg
# 电机控制消息

uint16 name
float64 kp       # 比例增益（力位混合模式使用）
float64 kd       # 微分增益（力位混合模式使用）
float64 pos      # 期望位置（rad）（位置模式和力位混合模式使用）
float64 spd      # 期望速度（rad/s）（位置模式、速度模式和力位混合模式使用）
float64 tor      # 前馈力矩（力位混合模式使用）
float64 cur      # 最大电流（A）（位置模式和速度模式使用）
string joint_ids  # 关节ID字符串，逗号分隔，如 "1,2,3"（标零模式使用）
```

命令行示例

代码块

```
# 腰部关节31使用力位混合控制
ros2 topic pub --once /waist/cmd ros2_bridge_msgs/msg/WaistCtrl "{
  header: {stamp: {sec: 0, nanosec: 0}, frame_id: 'waist'},
  mode: 1,
  label: 101,
  ctrl: [
    {name: 31, kp: 50.0, kd: 2.0, pos: 0.5, spd: 0, tor: 0}
  ]
}"


# 腰部关节31使用位置模式控制
ros2 topic pub --once /waist/cmd ros2_bridge_msgs/msg/WaistCtrl "{
  header: {stamp: {sec: 0, nanosec: 0}, frame_id: 'waist'},
  mode: 0,
  label: 101,
  ctrl: [
    {name: 31, pos: 0.5, spd: 0.2, cur: 10}
  ]
}"
```

4.1.3.4 头部关节

接口类型	topic
消息名称	/head/cmd
数据类型	ros2_bridge_msgs/msg/HeadCtrl
数据格式	代码块 ``` # HeadCtrl.msg # 头部关节控制消息 std_msgs/Header header uint8 mode # 控制模式：0=位置模式，1=力位混合模式，2=速度模式，3=标零模式，4=距离模式，5=电流模式 uint8 label # 调用标签（选填，或者填写101-255之间任意值） uint8 reserved # 保留字段，暂时未用 ros2_bridge_msgs/MotorCtrl[] ctrl ```
MotorCtrl	代码块 ``` # MotorCtrl.msg # 电机控制消息 uint16 name float64 kp # 比例增益（力位混合模式使用） float64 kd # 微分增益（力位混合模式使用） float64 pos # 期望位置（rad）（位置模式和力位混合模式使用） float64 spd # 期望速度（rad/s）（位置模式、速度模式和力位混合模式使用） float64 tor # 前馈力矩（力位混合模式使用） float64 cur # 最大电流（A）（位置模式和速度模式使用） string joint_ids # 关节ID字符串，逗号分隔，如 "1,2,3"（标零模式使用） ```

命令行示例	代码块
	``
	# 头部关节2使用力位混合控制
	ros2 topic pub --once /head/cmd ros2_bridge_msgs/msg/HeadCtrl "{
	  header: {stamp: {sec: 0, nanosec: 0}, frame_id: 'head'},
	  mode: 1,
	  label: 101,
	``

```
    ctrl: [
      {name: 2, kp: 50.0, kd: 2.0, pos: 0.5, spd: 0, tor: 0}
    ]
}"



# 头部关节2使用位置模式控制
ros2 topic pub --once /head/cmd ros2_bridge_msgs/msg/HeadCtrl "{
    header: {stamp: {sec: 0, nanosec: 0}, frame_id: 'head'},
    mode: 0,
    label: 101,
    ctrl: [
      {name: 2, pos: 0.5, spd: 0.2, cur: 10}
    ]
}"
```

4.2 灵巧手

4.2.1 启动说明

默认不启动灵巧手，如选配灵巧手，可通过诊断界面切换灵巧手配置，切换到对应的型号之后重启机器即可启动对应的手。

4.2.2 ros2接口

4.2.2.1 因时手

4.2.2.1.1 下发手指角度（13维）

接口类型	topic
消息名称	/right_hand/angle_set /left_hand/angle_set
数据类型	inspire_hand_msgs/msg/SetAngle

数据格式	代码块
	`#预留，暂无实际用途`
	`int32 hand_id`

```
# 0-12 分别表示各自由度对应的角度，单位0.1度，1200代表120度
# 0 小拇指指根，范围 900-1650
# 1 无名指指根，范围 900-1650
# 2 中指指根，范围 900-1650
# 3 中指测摆，范围 -150-150
# 4 食指指跟，范围 900-1650
# 5 食指测摆，范围 -150-150
# 6 小拇指指中，范围 1150-1900
# 7 无名指指中，范围 1150-1900
# 8 中指指中，范围 1150-1900
# 9 食指指中，范围 1150-1900
# 10 大拇指侧摆，范围 650-1750
# 11 大拇指一关节弯曲，范围 1250-1600
# 12 大拇指二关节弯曲，范围 1500-2040
int32[13] joint_values
```

命令行示例

代码块

```
# 握拳
ros2 topic pub -r 30 /right_hand/angle_set inspire_hand_msgs/msg/SetAngle "{joint_values: [1051,1051,1051,0,1051,0,1150,1150,1150,1150,1500,1400,1500]}"
```

4.2.2.1.2 设置力控阈值（13维）

接口类型	topic
消息名称	/right_hand/force_set /left_hand/force_set
数据类型	inspire_hand_msgs/msg/SetForce

数据格式	代码块
	`#预留，暂无实际用途`
	`int32 hand_id`
	`# 0-12 分别表示各自由度对应的力控阈值，单位g`
	`# 0 小拇指指根，范围 0-1000`
	`# 1 无名指指根，范围 0-1000`
	`# 2 中指指根，范围 0-1000`
	`# 3 中指测摆，范围 0-1000`
	`# 4 食指指跟，范围 0-1000`
	`# 5 食指测摆，范围 0-1000`

```
# 6 小拇指指中，范围 0-1000
# 7 无名指指中，范围 0-1000
# 8 中指指中，范围 0-1000
# 9 食指指中，范围 0-1000
# 10 大拇指侧摆，范围 0-1000
# 11 大拇指一关节弯曲，范围 0-1000
# 12 大拇指二关节弯曲，范围 0-1000
int32[13] joint_values
```

命令行示例

代码块

```
#力阈值控制
ros2 topic pub /left_hand/force_set
inspire_hand_msgs/msg/SetForce \
    "{joint_values:
[100,100,100,100,100,100,100,100,100,100,100,100,100]}
"
```

4.2.2.1.3 设置响应速度（13维）

接口类型	topic
消息名称	/right_hand/speed_set　/left_hand/speed_set
数据类型	inspire_hand_msgs/msg/SetSpeed
数据格式	代码块

```
#预留，暂无实际用途
int32 hand_id
# 0-12 分别表示各自由度对应的速度，速度为 2000：表示指手指在空载时从大角度运动到最小角度用时 1000ms，如果负载较大时，这个实际速度会有一定程度的降低
# 0 小拇指指根，范围 0-2500
# 1 无名指指根，范围 0-2500
# 2 中指指根，范围 0-2500
# 3 中指测摆，范围 0-2500
# 4 食指指跟，范围 0-2500
# 5 食指测摆，范围 0-2500
# 6 小拇指指中，范围 0-2500
# 7 无名指指中，范围 0-2500
# 8 中指指中，范围 0-2500
# 9 食指指中，范围 0-2500
# 10 大拇指侧摆，范围 0-2500
```

```
# 11 大拇指一关节弯曲，范围 0-2500
# 12 大拇指二关节弯曲，范围 0-2500
int32[13] joint_values
```

命令行示例	代码块
	`#速度值控制`
	`ros2 topic pub /left_hand/speed_set inspire_hand_msgs/msg/SetSpeed "{joint_values: [100,100,100,100,100,100,100,100,100,100,100,100,100]} "`

4.2.2.1.4 角度状态（13维）

接口类型	topic
消息名称	/right_hand/angle_actual /left_hand/angle_actual
数据类型	inspire_hand_msgs/msg/GetAngleAct
数据格式	代码块

```
#预留，暂无实际用途
int32 hand_id
# 0-12 分别表示各自由度对应的角度，单位0.1度，1200代表120度
# 0 小拇指指根，范围 900-1650
# 1 无名指指根，范围 900-1650
# 2 中指指根，范围 900-1650
# 3 中指测摆，范围 -150-150
# 4 食指指跟，范围 900-1650
# 5 食指测摆，范围 -150-150
# 6 小拇指指中，范围 1150-1900
# 7 无名指指中，范围 1150-1900
# 8 中指指中，范围 1150-1900
# 9 食指指中，范围 1150-1900
# 10 大拇指侧摆，范围 650-1750
# 11 大拇指一关节弯曲，范围 1250-1600
# 12 大拇指二关节弯曲，范围 1500-2040
int32[13] joint_values
# 固定值，描述自由度
#"little_root_bend", "ring_root_bend",
"middle_root_bend", "middle_side_swing",
"index_root_bend","index_side_swing",
"little_mid_bend",  "ring_mid_bend",
```

	"middle_mid_bend", "index_mid_bend","thumb_side_swing", "thumb_joint1_bend", "thumb_joint2_bend" 20 string[13] joint_names
命令行示例	代码块 1 #速度值控制 2 ros2 topic echo /left_hand/angle_actual

4.2.2.1.5 力反馈状态（13维）

接口类型	topic
消息名称	/right_hand/force_actual /left_hand/force_actual
数据类型	inspire_hand_msgs/msg/GetForceAct
数据格式	代码块 1 #预留，暂无实际用途 2 int32 hand_id 3 # 0-12 分别表示各自由度对应的力控阈值，单位g 4 # 0 小拇指指根，范围 0-1000 5 # 1 无名指指根，范围 0-1000 6 # 2 中指指根，范围 0-1000 7 # 3 中指测摆，范围 0-1000 8 # 4 食指指跟，范围 0-1000 9 # 5 食指测摆，范围 0-1000 10 # 6 小拇指指中，范围 0-1000 11 # 7 无名指指中，范围 0-1000 12 # 8 中指指中，范围 0-1000 13 # 9 食指指中，范围 0-1000 14 # 10 大拇指侧摆，范围 0-1000 15 # 11 大拇指一关节弯曲，范围 0-1000 16 # 12 大拇指二关节弯曲，范围 0-1000 17 int32[13] joint_values 18 # 固定值，描述自由度 19 #"little_root_bend", "ring_root_bend", "middle_root_bend", "middle_side_swing", "index_root_bend","index_side_swing", "little_mid_bend", "ring_mid_bend", "middle_mid_bend",

```
    "index_mid_bend","thumb_side_swing",
    "thumb_joint1_bend", "thumb_joint2_bend"
20  string[13] joint_names
```

命令行示例	代码块
	`#力阈值控制`
	`ros2 topic echo /left_hand/force_actual`

4.2.2.1.6 触觉反馈状态

接口类型	topic
消息名称	/right_hand/touch_data /left_hand/touch_data
数据类型	inspire_hand_msgs/msg/TouchData
数据格式	代码块

```
#0 小拇指指尖法向力 单位0.01N
#1 无名指指尖法向力
#2 中指指尖法向力
#3 食指指指尖法向力
#4 大拇指指尖法向力
int32[5] finger_forces
#0 小拇指指尖切向力 单位0.01N
#1 无名指指尖切向力
#2 中指指尖切向力
#3 食指指指尖切向力
#4 大拇指指尖切向力
int32[5] finger_tangentials
#0 小拇指指尖切向力方向 单位角度（顺时针360度，无接触时返回65535）
#1 无名指指尖切向力方向
#2 中指指尖切向力方向
#3 食指指指尖切向力方向
#4 大拇指指尖切向力方向
int32[5] finger_angles
#0 小拇指指尖接近觉（0-16000000）
#1 无名指指尖接近觉
#2 中指指尖接近觉
#3 食指指指尖接近觉
#4 大拇指指尖接近觉
int32[5] finger_proximity
```

```
# plam_data1-3掌左 法向力，切向力，切向力方向
# plam_data4-6掌中，plam_data7-9掌右
int32[9] plam_data
```

命令行示例

代码块

```
#力阈值控制
ros2 topic echo /left_hand/touch_data
```

4.2.2.2 强脑手

4.2.2.2.1 下发控制（1维）

接口类型	topic
消息名称	/right_hand/set_motor_single /left_hand/set_motor_single
数据类型	brainco_hand_msgs::msg::SetMotorSingle

数据格式	代码块
	`uint8 mode  # 1~6，对应以下6种控制模式`
	`# 1. 位置`
	`# 2. 速度`
	`# 3. 电流`
	`# 4. PWM`
	`# 5. 位置+期望时间`
	`# 6. 位置+期望速度`
	
	`uint8 motor_id  # 手指ID 1-6 拇指弯曲，拇指旋转，食指-小指`
	`uint16 position  # 位置  1-1000`
	`int16 speed  # 速度  -1000 - 1000`
	`int16 current  # 电流  -1000 - 1000`
	`int16 pwm  # PWM  -1000 - 1000`
	`uint16 duration  # 期望时间，单位ms 1~2000`

命令行示例	代码块 `# 单指弯曲`

```
ros2 topic pub --once /right_hand/set_motor_single
brainco_hand_msgs/msg/SetMotorSingle "{mode: 1,
motor_id: 3, position: 800}"
```

4.2.2.2.2 下发控制（6维）

接口类型	topic
消息名称	/right_hand/set_motor_multi /left_hand/set_motor_multi
数据类型	brainco_hand_msgs::msg::SetMotorMulti

数据格式	代码块
	`uint8 mode # 1~6, 对应以下6种控制模式`
	`# 1. 位置`
	`# 2. 速度`
	`# 3. 电流`
	`# 4. PWM`
	`# 5. 位置+期望时间`
	`# 6. 位置+期望速度`
	`# 索引 手指ID 0-6 拇指弯曲, 拇指旋转, 食指-小指`
	`uint16[6] positions # 位置 1-1000`
	`int16[6] speeds # 速度 -1000 - 1000`
	`int16[6] currents # 电流 -1000 - 1000`
	`int16[6] pwm # PWM -1000 - 1000`
	`uint16[6] durations # 期望时间, 单位ms 1~2000`

命令行示例	代码块
	`# 握拳`
	`ros2 topic pub -r 30 /right_hand/set_motor_multi brainco_hand_msgs/msg/SetMotorMulti "{mode: 1, positions: [100, 200, 300, 400, 500, 600]}"`

4.2.2.2.3 手指状态（6维）

接口类型	topic
消息名称	/right_hand/motor_status /left_hand/motor_status

数据类型	brainco_hand_msgs::msg::MotorStatus

数据格式	代码块
	`uint16[6] positions  #位置  1-1000`
	`int16[6] speeds  #速度  -1000 - 1000`
	`int16[6] currents  #电流  -1000 - 1000`
	`uint8[6] states  #状态`

命令行示例	代码块
	`# 握拳`
	`ros2 topic echo /left_hand/motor_status`

4.2.2.2.4 触觉反馈状态

接口类型	topic
消息名称	/right_hand/touch_status /left_hand/touch_status
数据类型	brainco_hand_msgs::msg::TouchStatus
数据格式	代码块

```
#brainco_hand_msgs::msg::TouchStatus
TouchStatusItem[5] data    #状态数组

#brainco_hand_msgs::msg::TouchStatusItem
uint16 normal_force1          #法向力 数值单位是 100 * N
uint16 normal_force2 #预留
uint16 normal_force3 #预留
uint16 tangential_force1        #切向力 数值单位是 100 * N
uint16 tangential_force2 #预留
uint16 tangential_force3 #预留
uint16 tangential_direction1    #切向力方向，单位是角度，数值范围为 0 ~ 359 度。靠近指尖的方向为 0 度，按顺时针旋转最大到 359 度，当数值为 65535 (0xFFFF) 时，表示切向力方向无效
uint16 tangential_direction2 #预留
uint16 tangential_direction3 #预留
uint32 self_proximity1          #接近值
uint32 self_proximity2 #预留
uint32 mutual_proximity #预留
uint16 status                   #触觉状态寄存器为 16 位，分高低字节表示不同状态信息：  低 8 位表示整体数据状态：0 表示
```

	数据正常，1 表示数据异常，2 表示与触觉传感器通信异常。 高 8 位为数据异常志位： #Bit0 表示原始值错误， #Bit1 表示原始值长时间未更新， #Bit2 表示触发超时。

命令行示例	代码块
	`# 握拳`
	`ros2 topic echo  /left_hand/touch_status`

4.2.2.2.5 获取设备信息

接口类型	service
消息名称	/left_hand/get_device_info /right_hand/get_device_info
数据类型	brainco_hand_msgs:srv:GetDeviceInfo

数据格式	代码块
	`# 请求部分`
	`bool get_voltage  # 是否获取电池电压`
	`bool get_turbo_mode  # 是否获取配置信息（turbo模式）`
	`---`
	`# 响应部分`
	`bool success`
	`# 操作是否成功`
	`uint8 sku_type`
	`# 设备 SKU 类型`
	`string serial_number`
	`# 设备序列号`
	`string firmware_version`
	`# 固件版本`
	`uint16 voltage`
	`# 电池电压`
	`bool turbo_mode  # 是否处于 turbo 模式`

命令行示例	代码块 `ros2 service call /left_hand/get_device_info brainco_hand_msgs/srv/GetDeviceInfo \`

```
  "{get_voltage: true, get_turbo_mode: true}"
```

4.2.2.2.6 下发配置（6维）

接口类型	service
消息名称	/left_hand/set_motor_multi /right_hand/set_motor_multi
数据类型	brainco_hand_msgs:srv:SetMotorMulti

数据格式	代码块
	`#7 设置 6 指保护电流`
	`#8 重置触觉传感器`
	`#9 校准触觉传感器`
	`#10 开启 Turbo 模式`
	`uint8 mode`
	
	`uint16[6]  positions  # 无效`
	`int16[6]  speeds  # 无效`
	`int16[6]  currents  # 电流保护值范围 100~1500 mA，默认 500`
	`int16[6]  pwms  # 无效`
	`PWMuint16[6]  durations  # 无效`
	`---`
	`bool success  # 执行结果`

命令行示例	代码块
	`# 握拳`
	`# mode 7: 设置 6 指保护电流为 500mA`
	`ros2 service call /left_hand/set_motor_multi brainco_hand_msgs/srv/SetMotorMulti \`
	`  "{mode: 7, positions: [0,0,0,0,0,0], speeds: [0,0,0,0,0,0], \`
	`  currents: [100,100,100,100,100,100], pwms: [0,0,0,0,0,0], durations: [0,0,0,0,0,0]}"`
	
	`# mode 8: 重置全部触觉传感器`
	`ros2 service call /right_hand/set_motor_multi brainco_hand_msgs/srv/SetMotorMulti \`
	`  "{mode: 8, positions: [0,0,0,0,0,0], speeds: [0,0,0,0,0,0], \`

```
    currents: [0,0,0,0,0,0], pwms: [0,0,0,0,0,0], durations:
[0,0,0,0,0,0]}"

# mode 9: 校准全部触觉传感器
ros2 service call /right_hand/set_motor_multi
brainco_hand_msgs/srv/SetMotorMulti \
  "{mode: 9, positions: [0,0,0,0,0,0], speeds: [0,0,0,0,0,0],
\
    currents: [0,0,0,0,0,0], pwms: [0,0,0,0,0,0], durations:
[0,0,0,0,0,0]}"

# mode 10: 开启 Turbo 模式
ros2 service call /left_hand/set_motor_multi
brainco_hand_msgs/srv/SetMotorMulti \
  "{mode: 10, positions: [0,0,0,0,0,0], speeds:
[0,0,0,0,0,0], \
    currents: [0,0,0,0,0,0], pwms: [0,0,0,0,0,0], durations:
[0,0,0,0,0,0]}"
```

4.2.2.2.7 下发配置（1维）

接口类型	service
消息名称	/left_hand/set_motor_single /right_hand/set_motor_single
数据类型	brainco_hand_msgs:srv:SetMotorSingle

数据格式	代码块
	`#7 设置 单指保护电流`
	`uint8 mode`
	
	`uint8 motor_id  # 手指ID 1-6 拇指弯曲，拇指旋转，食指-小指`
	`uint16  positions  # 无效`
	`int16  speeds  # 无效`
	`int16  currents  # 电流保护值范围 100~1500 mA，默认 500`
	`int16  pwms  # 无效`
	`PWMuint16  durations  # 无效`
	`---`
	`bool success  # 执行结果`

命令行示例	代码块

```
# mode 7: 设置拇指（motor_id=1）保护电流为 600mA
ros2 service call /left_hand/set_motor_single
brainco_hand_msgs/srv/SetMotorSingle \
  "{mode: 7, motor_id: 1, position: 0, speed: 0, current:
600, pwm: 0, duration: 0}"

# mode 7: 设置食指（motor_id=3）保护电流为 500mA
ros2 service call /right_hand/set_motor_single
brainco_hand_msgs/srv/SetMotorSingle \
  "{mode: 7, motor_id: 3, position: 0, speed: 0, current:
500, pwm: 0, duration: 0}"
```

5. 传感器

5.1 IMU

接口类型	topic
消息名称	/robot_state
数据类型	ros2_bridge_msgs/msg/RobotState

整体数据格式	代码块
	`std_msgs/Header header`
	
	`ros2_bridge_msgs/HeadStatus head`
	`ros2_bridge_msgs/WaistStatus waist`
	`ros2_bridge_msgs/LegStatus leg`
	`ros2_bridge_msgs/ArmStatus arm`
	`ros2_bridge_msgs/ImuStatus imu`

imu数据格式	代码块
	`# ImuStatus.msg`
	`# IMU 状态消息`
	
	`std_msgs/Header header`
	`uint8 label # 调用标签`
	`uint8 reserved # 保留字段，暂时未用`
	
	`# 四元数`
	`float64 qx`

```
float64 qy
float64 qz
float64 qw

# 欧拉角（度）
float64 roll
float64 pitch
float64 yaw

# 角速度（rad/s）
float64 wx
float64 wy
float64 wz

# 线性加速度（m/s^2）
float64 ax
float64 ay
float64 az

# 协方差矩阵（按行展开）
float64[9] orientation_covariance
float64[9] angular_velocity_covariance
float64[9] linear_acceleration_covariance

# 错误码
uint32 error
```

命令行示例

代码块

```
ros2 topic echo /robot_state
```

5.2 相机

5.2.1 启动说明

相机默认不启动，如需使用以系统服务的形式启动，头部与腰部的摄像头的启动方法分别为，在算力主机（192.168.41.2）上运行

代码块

```
sudo systemctl start orbbec_head.service
sudo systemctl start orbbec_waist.service
```

关闭方法为

```
sudo systemctl stop orbbec_head.service
sudo systemctl stop orbbec_waist.service
```

如需设置为开机启动，运行

```
sudo systemctl enable orbbec_head.service
sudo systemctl enable orbbec_waist.service
```

5.2.2 ros2接口

头部和腰部相机的ros2 topic 为

```
#头部相机
/ob_camera_head/color/camera_info       #彩色相机内参
/ob_camera_head/color/image_raw         #彩色原始图像（默认RGB8 格式，分辨率1280×720）
/ob_camera_head/color/metadata          #彩色图像元数据
/ob_camera_head/depth/camera_info       #深度相机内参
/ob_camera_head/depth/image_raw         #深度原始图像
/ob_camera_head/depth/metadata          #深度图像元数据
/ob_camera_head/depth_to_color          #深度相机→彩色相机的坐标变换
#腰部相机
/ob_camera_waist/color/camera_info      #彩色相机内参
/ob_camera_waist/color/image_raw        #彩色原始图像（默认 RGB8 格式，分辨率1280×720）
/ob_camera_waist/color/metadata         #彩色图像元数据
/ob_camera_waist/depth/camera_info      #深度相机内参
/ob_camera_waist/depth/image_raw        #深度原始图像
/ob_camera_waist/depth/metadata         #深度图像元数据
/ob_camera_waist/depth_to_color         #深度相机→彩色相机的坐标变换
```

5.2.3 相机参数修改

头部相机参数可修改 xos/orbbec_camera/share/orbbec_camera/launch/head_330_tg30_thor.launch.py，腰部可修改 xos/orbbec_camera/share/orbbec_camera/launch/waist_330_tg30_thor.launch.py.

具体修改方法为，在lauch_arguments中，增加想要修改的参数名和值，具体可修改的参数可参考相机启动参数

5.3 雷达

5.3.1 获取雷达传感器点云信息

接口类型	topic
消息名称	/livox/lidar
频率	10hz
数据类型	sensor_msgs::msg::PointCloud2

数据格式	代码块
	`float32 x # X axis, unit:m`
	`float32 y # Y axis, unit:m`
	`float32 z # Z axis, unit:m`
	`float32 intensity # the value is reflectivity, 0.0~255.0`
	`uint8 tag # livox tag`
	`uint8 line # laser number in lidar`
	`float64 timestamp # Timestamp of point`

5.3.2 获取雷达传感器IMU信息

接口类型	topic
消息名称	`/livox/imu`
频率	200hz
数据类型	`sensor_msgs::msg::Imu`

数据格式	代码块
	`std_msgs/Header header`
	`geometry_msgs/Quaternion orientation`
	`float64[9] orientation_covariance`
	`geometry_msgs/Vector3 angular_velocity`
	`float64[9] angular_velocity_covariance`
	`geometry_msgs/Vector3 linear_acceleration`

```
7   float64[9] linear_acceleration_covariance
```

5.4 室外卫星导航套装(选配)

- 说明：获取GPS数据信息。

接口类型	topic
消息名称	`/gps/fix`
频率	1hz
数据类型	`navigation_msgs/msg/GpsFix`
数据格式	代码块 1 std_msgs/Header header 2 float64 latitude #纬度 3 float64 longitude #经度 4 float64 altitude #高度 5 int32 status #定位状态 0:定位不可用或无效 1:单点定位 2:伪距差分或SBAS定位 4:RTK固定解 5:RTK浮点解 6 int32 num_sats #卫星数量 7 float64 hdop #定位精度 8 float64 speed #速度 m/s 9 float64 heading #航向角 10 float64 tx #GPS和PPS时间差 T(GPS)-T(PPS) ms

5.5 六维力

接口类型	topic
消息名称	/left_hand/dof6_force /right_hand/dof6_force
数据类型	geometry_msgs::msg::WrenchStamped
数据格式	代码块 1 std_msgs/Header header 2 Wrench wrench 3 4 Vector3 force #三个方向的力

```
5   Vector3  torque  #三个方向的力矩
```

命令行示例	代码块 `1  ros2 topic echo /right_hand/dof6_force`

6. 通信模块

6.1 蓝牙

系统软件提供蓝牙gatt服务，gatt客户端可使用系统蓝牙服务获取机器人序列号、x86平台的ip、x86平台的wifi模式、对x86配网等操作。

6.1.1 接口定义

6.1.1.1 获取WIFI IP地址

UUID	00000002-1E3D-FAD4-74E2-97A033F1BF00
类型	"broadcast", "notify", "read"
返回值	ip地址（如：192.168.31.1）

6.1.1.2 获取设备SN

UUID	00000004-1E3D-FAD4-74E2-97A033F1BF00
类型	read
返回值	sn（如：00TG20PD25200004）

6.1.1.3 获取设备网络模式

UUID	"00000005-1E3D-FAD4-74E2-97A033F1BF00
类型	"read", "notify"
返回值（字符串）	“AP”或“managed”

6.1.1.4 请求连接WIFI

UUID	00000003-1E3D-FAD4-74E2-97A033F1BF00
类型	"read", "write", "notify"
请求	{"command":"CMD_CONNECT_WIFI","ssid":"优必选机器人","password":"88888888"}
成功响应	{"error_code":"ERROR_CODE_SUCCESS", "error_message":"设备 \"wlo1\" 成功以 \"06b8d1de-28bb-46a5-83e9-835c1b34c25e\" 激活"}
失败响应	{"error_code":"ERROR_CODE_CONNECT_WIFI_FAILED", "error_message":"错误：未发现 SSID 为 "优必选机器人" 的网络"}

6.1.1.5 切换网络模式

UUID	00000006-1E3D-FAD4-74E2-97A033F1BF00
类型	"write", "notify"
请求	“AP”或“managed”
成功响应	“success”
失败响应	“false”

6.2 5G模块

5G模块默认不启动，如需使用以系统服务的形式启动，启动方法为，在算力主机（192.168.41.2）上运行

代码块

```
sudo systemctl start init_5G.service
```

5G模块需要一定的时间拨号获得网络，可以使用 `ifconfig` 观察 `eno3` 网卡，获取到ipv4的地址时说明网络连接建立成功。

关闭时停止服务

代码块

```
sudo systemctl stop init_5G.service
```

其他与5G相关的功能如切换供应商，查看卡号等，可查询《天工行者DEX Pro版用户手册》。

7. 语音

7.1 使用方式

语音功能目前包含识别、语义输出、tts合成、音频播放模块。该模块通过识别有效音视频，进行意图识别、语义输出和播音等功能，达到能够与人智能对话的目的

7.1.1 启动方式

lyre的启动方式是通过proc_manager进程进行管理，当对proc_manager进行正常启动30s后，lyre即可正常启动

7.1.2 使用方式

通过遥控器上拨F键后，然后长按A即可开启语音对话，再次长按A键后，即可关闭对话

7.2 ros2接口

7.2.1 lyre 类型接口

7.2.1.1 ASR文本

用于输出人脸唤醒、关键词唤醒、vadstart、vadend、asr等数据

接口类型	topic
消息名称	/lyre/voice_activity
数据类型	lyre_msgs/msg/LyreVoiceActivity
数据格式	代码块 1 # lyre_msgs/msg/LyreVoiceActivity.msg 2 # ASR 语音识别到的内容。 3 4 string TOPIC_NAME = /lyre/voice_activity 5 6 string content

命令行示例	代码块 1 ros2 topic echo /lyre/voice_activity
协议具体内容	代码块 1 **人脸识别唤醒 ： eventType: 20** 2 content: '{"content": {"eventType":20,"info":"mock_event"},"timestamp":"1774591634 .807","traceId":"a044418b-555d-46f5-ae61- 5875fe3ea4b0","type":"aiui_event"}' 3 4 **关键词唤醒： eventType: 4** 5 content: '{"content":{"eventType":4,"result":{"ivw": {"angle":93}}},"timestamp":"1774592106.879","traceId":"2f02c f62-86ad-4d66-898a-0c90d8f16930","type":"aiui_event"}' 6 7 **退出对话: eventType: 5** 8 content: '{"content": {"eventType":5,"info":"mock_event"},"timestamp":"1774591634. 666","traceId":"6710113f-80f2-4add-8bff- 8c039438e63e","type":"aiui_event"}' 9 10 **vadstart: eventType: 6， arg1:0** 11 content: '{"content": {"arg1":0,"eventType":6,"info":"mock_vad"},"timestamp":"1774 592139.888","traceId":"61332d9e-b0dc-4a6d-ae6a- e52ea17a08e3","type":"aiui_event"}' 12 **vadend: eventType: 5 arg1:2** 13 content: '{"content": {"arg1":2,"eventType":6,"info":"mock_vad"},"timestamp":"1774 592140.924","traceId":"c622a4f3-cacd-44aa-b1b4- e7d42b683c0f","type":"aiui_event"}' 14 15 **asr返回结果： eventType: 1** 16 content: '{"content":{"eventType":1,"result":{"text":{"ws": [{"cw":[{"w":"今天的天 气"}]}]}}},"timestamp":"1774592141.073","traceId":"53583960- 0596-4f2a-a858-b7d8957fda6d","type":"aiui_event"}'

7.2.1.2 拾音器原始音频流

用于输出原始音频

接口类型	topic
消息名称	/lyre/audio_stream
数据类型	lyre_msgs/msg/AudioFrame

数据格式	代码块
	`# 音频帧消息`
	
	`string TOPIC_NAME = /lyre/audio_stream`
	
	`uint32 sample_rate  # 采样率 (Hz)`
	`uint8 channels  # 声道数 (1=单声道, 2=立体声)`
	`uint8 bits_per_sample  # 位深度 (8/16/24/32)`
	`uint32 sequence  # 帧序号`
	`uint32 timestamp_sec  # 时间戳 (秒)`
	`uint32 timestamp_nsec  # 时间戳 (纳秒)`
	`uint8[] data  # 音频数据`

命令行示例	代码块 `ros2 topic echo /lyre/audio_stream`

7.2.1.3 拾音音频流开关

用于控制是否打开原始音频输出

接口类型	service
消息名称	/lyre/audio_control
数据类型	lyre_msgs/srv/AudioControl

数据格式	代码块
	`#`
	`============================================================`
	`# AudioControl.srv - 音频流控制服务`
	`#`
	`============================================================`

```
===
#
# 【服务名称】 /lyre/audio_control
# 【服务类型】 lyre_msgs/srv/AudioControl
#
# ==================== 使用方法 ====================
#
# 命令行调用:
#   ros2 service call /lyre/audio_control
lyre_msgs/srv/AudioControl "{enable: true}"
#
# ==================== 参数说明 ====================
#
# 请求 (Request):
#   bool enable  - true: 启动, false: 停止
#
# 响应 (Response):
#   bool success    - true: 成功, false: 失败
#   string message  - 返回信息
#
# ==================== 相关话题 ====================
#
# 音频数据话题: /lyre/audio_stream
# 消息类型: lyre_msgs/msg/AudioFrame
#
#
============================================================
===
bool enable
---
bool success
string message

```

命令行示例

代码块

```
ros2 service call /lyre/audio_control
lyre_msgs/srv/AudioControl "{enable: true}"
```

7.2.2 LLM 输入

用于将ASR 结果或图像等事件做为大模型推理的输入，下发给 LLM 推理节点。

接口类型	topic
消息名称	/intelligent_interaction/llm/input
数据类型	interaction_msgs/msg/LlmInput
数据格式	代码块 1 # LlmInput.msg 2 string sid # 本次会话 sessionId，与上游 ASR/事件的 traceId 保持一致 3 string trigger_type # 触发方式，当前支持："asr" 4 string prompt_template # 提示词模板，含占位符，如 "{asr}" 5 string params # 模板渲染参数（JSON 字符串），如 {"asr":"今天天气怎么样"} 6 int32 priority # 优先级；asr 触发请置 1
命令行示例	代码块 1 ros2 topic pub --once /intelligent_interaction/llm/input \ 2 interaction_msgs/msg/LlmInput \ 3 "{sid: 'demo-001', trigger_type: 'asr', prompt_template: '{asr}', params: '{\"asr\":\"你好DEX\"}', priority: 1}"

7.2.3 智能交互信息回复

LLM 推理节点输出的流式回复文本。用于供 TTS 节点订阅并合成语音。

接口类型	topic
消息名称	/intelligent_interaction/llm/rst
数据类型	interaction_msgs/msg/LlmRst
数据格式	代码块 1 # LlmRst.msg 2 string TOPIC_NAME = /intelligent_interaction/llm/rst 3 string sid # 本次回复的 sessionId；同一轮回复的所有 chunk 必须使用同一个 sid 4 uint32 seq # 序列号；流式回复从 1 开始，每个 chunk 严格 +1；非流式固定为 1 5 bool last # 是否为本轮最后一帧

```
6   string text                    # 本帧文本内容
```

命令行示例

代码块

```
1   ros2 topic pub --once /intelligent_interaction/llm/rst \
2       interaction_msgs/msg/LlmRst \
3       "{sid: 'demo-001', seq: 1, last: false, text: '你好，'}"
```

7.2.4 交互开启/关闭/查询

用于开启/关闭或查询机器人当前是否处于"可交互"状态。上层应用通过本接口控制对话功能。

接口类型	service
消息名称	/intelligent_interaction/interactive/state
数据类型	interaction_msgs/srv/InstructionService

数据格式	代码块
	`# InstructionService.srv`
	`string cmd  # 指令：start / stop / query`
	`---`
	`bool success  # cmd=start/stop：是否切换成功`
	`# cmd=query ：true 已开启 / false 已关闭`

命令行示例	代码块
	`# 开启交互`
	`ros2 service call /intelligent_interaction/interactive/state \`
	`interaction_msgs/srv/InstructionService "{cmd: 'start'}"`

7.2.5 TTS/语音播报/播报查询

非流式 TTS 主入口：用于将一段文本 / 本地音频 / 远端 URL 排队播报或打断当前播报或查询播报状态。

接口类型	service

消息名称	/intelligent_interaction/tts/play
数据类型	interaction_msgs/srv/TtsService
数据格式	代码块 ``` # TtsService.srv string text # 文本 / 本地音频文件绝对路径 / URL; cmd=query 时本字段无效 string type # 资源类型: text / file / url; cmd=query 时本字段无效 string cmd # 指令: append / stop / query string audio_id #音频播放优先级id string extra #扩展字段 --- bool success # 指令是否成功执行 string status # cmd=query 时返回: "idle" 空闲 / "playing" 播报中 / "none" 未知 # 其余 cmd 通常为空字符串或 "none"; 执行失败时携带异常信息 ```
命令行示例	代码块 ``` # 打断当前播报并播放一段文本 ros2 service call /intelligent_interaction/tts/play \ interaction_msgs/srv/TtsService \ "{text: '你好我是天工', type: 'text', cmd: 'stop', audio_id: '', extra: ''}" # 按 audio_id 播放配置里的音频 ros2 service call /intelligent_interaction/tts/play \ interaction_msgs/srv/TtsService \ "{audio_id: 'audio_chenggong', cmd: 'play', type: '', text: '', extra: ''}" # stop-and-play 指定文件, 并使用 audio_chenggong 的配置优先级 ros2 service call /intelligent_interaction/tts/play \ interaction_msgs/srv/TtsService \ "{audio_id: 'audio_chenggong', type: 'file', text: '/home/nvidia/data/speech/builtin/chenggong.wav', cmd: 'stop', extra: ''}" #是否停止取决于audio_id的优先级 ros2 service call /intelligent_interaction/tts/play \ interaction_msgs/srv/TtsService \ ```

```
19    "{audio_id: 'audio_chenggong', cmd: 'stop', type: '',
    text: '', extra: ''}"
```

7.2.6 流式文本播报

TTS节点对7.2.3中**/intelligent_interaction/llm/rst**进行订阅消费，用于合成语音。

接口类型	topic
消息名称	/intelligent_interaction/llm/rst
数据类型	interaction_msgs/msg/LlmRst

数据格式	代码块
	`# LlmRst.msg`
	`string TOPIC_NAME = /intelligent_interaction/llm/rst`
	`string sid  # 本次回复的 sessionId；同一轮回复的所有 chunk 必须使用同一个 sid`
	`uint32 seq  # 序列号；流式回复从 1 开始，每个 chunk 严格 +1；非流式固定为 1`
	`bool  last  # 是否为本轮最后一帧`
	`string text  # 本帧文本内容`

命令行示例	代码块
	`ros2 topic pub --once /intelligent_interaction/llm/rst \`
	`  interaction_msgs/msg/LlmRst \`
	`  "{sid: 'demo-001', seq: 1, last: false, text: '你好，'}"`

7.2.7 流式音频流

用于将外部已合成的 PCM 音频流传递给 TTS 节点，进入播放队列。

接口类型	topic
消息名称	/intelligent_interaction/audio/stream
数据类型	interaction_msgs/msg/AudioStream
数据格式	代码块 `# AudioStream.msg`

```
string  sid                 # 音频流 sessionId；同一轮所有 chunk 共用
uint32  seq                 # 序列号；从 1 开始严格 +1；非流式固定为 1
uint8   channels            # 声道数（1=单声道，2=立体声）
uint32  sample_rate         # 音频采样率（Hz）
string  encoding "S16LE"    # 编码格式，当前固定为 S16LE（16bit 小端）
bool    last                # 是否为本轮最后一帧
uint8[] data                # PCM 数据；长度需满足 channels × 2 字节对齐
```

命令行示例

代码块

```
ros2 topic pub --once /intelligent_interaction/audio/stream \
    interaction_msgs/msg/AudioStream \
    "{sid: 'pcm-001', seq: 1, channels: 1, sample_rate: 24000, encoding: 'S16LE', last: true, data: [0,0,0,0]}"
```

7.2.8 音色更新

用于在线切换 TTS 音色 ID。

接口类型	service
消息名称	/intelligent_interaction/tts/update_voice_id
数据类型	interaction_msgs/srv/InstructionService

数据格式	代码块
	`# InstructionService.srv`
	`string cmd  # 目标音色 ID`
	`  # 当前支持：tiangong_v1 ~ tiangong_v5`
	`---`
	`bool  success  # true 更新成功 / false 失败（非法 voice_id 或写盘异常）`

命令行示例	代码块 `ros2 service call /intelligent_interaction/tts/update_voice_id \`

```
2       interaction_msgs/srv/InstructionService "{cmd:
    'tiangong_v3'}"
```